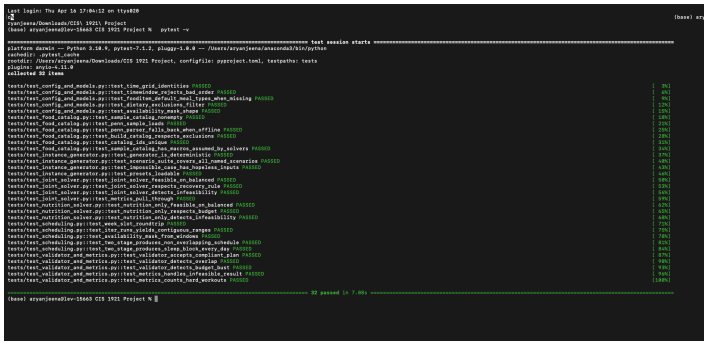
Group Members: Aryan Jeena, Aadithya Srinivasan
Project Title: Constraint-Based Training, Nutrition, and Health Schedule Optimizer

Progress:

The project runs end to end and is roughly eighty percent done. We have a typed domain model, three solver formulations behind a shared interface, a food catalog with a Penn Dining fallback and USDA support, a parameterized instance generator, a passing test suite, a batch experiment runner, and a Streamlit UI. Most of what is left is analysis and write-up rather than new modeling work. Every quantity the solvers see is an integer (minutes, cents, grams, and 30-minute slots), both because CP-SAT requires integer coefficients and because it makes our tests much easier to reason about. The calendar is a 7 by 48 grid, so one week of planning is 336 slots. The three solvers are the heart of the project. The first, `nutrition_only`, is a pure MIP that decides only how many integer servings of each food to eat on each day. The second, `two_stage`, runs that same MIP and then hands its serving decisions to a CP-SAT scheduler that places meals, workouts, and sleep into the user's available time windows. The third, `joint_cpsat`, is a single CP-SAT model that decides servings, placements, and recovery spacing all at once. Keeping all three compatible lets us directly answer one of the questions from the proposal, namely when joint reasoning beats a two-stage pipeline. Hard and soft constraints stay clearly separated in the code. The hard side covers the calorie band, the daily protein floor, the weekly budget, dietary exclusions, availability windows, non-overlap, the workout count range, a minimum sleep block, and recovery gaps between hard workouts. The soft side lives in the objective and covers protein shortfall, absolute deviation of carbs and fat from target, total cost, avoid-day violations, preferred-day bonuses, peri-workout meal timing, and a small convenience term.

On data, we ship a 47-item curated catalog along with an 8-item Penn Dining sample so the project runs out of the box, with USDA FoodData Central wired in as a third source. The instance generator produces nine synthetic users by default, and the last of them, `impossible_case`, has deliberately contradictory constraints so we can confirm that every solver detects infeasibility rather than silently returning a broken plan. Six JSON presets in `configs/presets/` plus the Streamlit sidebar form both let users edit constraints without touching code. All 32 of our unit tests pass in under 8 seconds. The suite covers config and domain validation, food-catalog filtering, feasibility and infeasibility detection for every solver, non-overlap on the scheduled week, budget enforcement, recovery spacing, and a roundtrip of the slot grid.



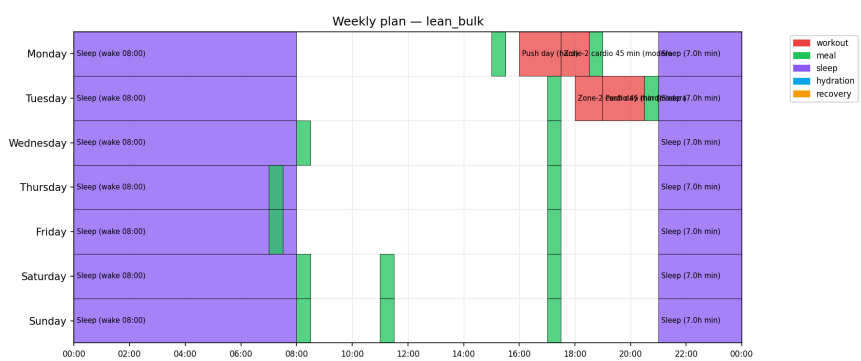
To show that the three formulations are directly comparable, we ran `python -m src.app.cli demo --time-limit 15` on an auto-generated balanced user carrying 56 foods and 10 workouts. All three solvers returned `OPTIMAL`. Nutrition-only finished in 0.08 seconds, two-stage in 0.19, and joint CP-SAT in

0.69. The joint solver is slowest because it reasons over meal intervals across 56 foods plus optional workout intervals plus sleep blocks in one model, and that ordering is exactly what we expected. Nutrition-only reports zero workouts scheduled, which is not a bug but the point of the comparison: that model simply does not place workouts, and that is part of why the more expensive joint model is worth its cost.

```
(base) aryanjeena@lev-15663 CIS 1921 Project %   python -m src.app.cli demo --time-limit 15

Demo run on scenario='balanced_1921' (56 foods, 10 workouts)

--- nutrition_only ---
status=OPTIMAL runtime=0.08s objective=-266.0
cal_dev_abs=280, cost=0c, workouts=0, preferred_day_hits=0

--- two_stage ---
status=OPTIMAL runtime=0.19s objective=0.0
cal_dev_abs=280, cost=0c, workouts=3, preferred_day_hits=3

--- joint_cpsat ---
status=OPTIMAL runtime=0.69s objective=-266.0
cal_dev_abs=280, cost=0c, workouts=3, preferred_day_hits=3

(base) aryanjeena@lev-15663 CIS 1921 Project %
```

Running `python -m src.app.cli solve --preset lean_bulk --solver joint_cpsat --time-limit 20 --figure` returned `OPTIMAL` in 1.14 seconds with an objective of -203. The resulting plan has four workouts, two of them hard and separated by the required recovery gap, and averages 3120 kcal, 202 g protein, 368 g carbs, and 92 g fat per day. Weekly cost came out to zero cents because the solver stacked Penn Dining swipe items, which our catalog prices at zero since they are already covered by the meal plan. The `--figure` flag wrote the schedule to `reports/figures/schedule_lean_bulk_joint_cpsat.png`.

```
(base) aryanjeena@lev-15663 CIS 1921 Project %   python -m src.app.cli solve --preset lean_bulk --solver joint_cpsat --time-limit 20 --figure

Solving with joint_cpsat on lean_bulk (56 foods, 10 workouts)...
Status: OPTIMAL  runtime=1.14s  objective=-203.0

Metrics:
  feasible: True
  solver: joint_cpsat
  status: OPTIMAL
  runtime_s: 1.1362
  objective_value: -203.0
  workouts_scheduled: 4
  hard_workouts_scheduled: 2
  avg_calories: 3120.0
  avg_protein_g: 202.0
  avg_carbs_g: 368.0
  avg_fat_g: 92.0
  total_cost_cents: 0
  calorie_deviation_abs: 840
  protein_gap_to_target_g: 0
  part_workout_meal_hits: 4
  preferred_day_hits: 4
  avoid_day_violations: 0

Weekly plan for lean_bulk
  Monday
    00:00-08:00 [sleep] Sleep (wake 08:00)
    15:00-15:30 [meal] snack: 1920 Commons: Greek Yogurt Parfait x2
    16:00-17:30 [workout] Push day (hard)
    17:30-18:30 [workout] Zone-2 cardio 45 min (moderate)
    18:30-19:00 [meal] dinner: 1920 Commons: Chicken & Rice Bowl x2, Hill House: Mediterranean Veggie Bowl x1, KCECH: Salmon with Grains x1, New College House: Tofu Stir-fry x1
    21:00-08:00 [sleep] Sleep (7.0h min)
  Tuesday
    00:00-08:00 [sleep] Sleep (wake 08:00)
    17:00-17:30 [meal] dinner: 1920 Commons: Chicken & Rice Bowl x2, Hill House: Mediterranean Veggie Bowl x1, KCECH: Salmon with Grains x1, New College House: Tofu Stir-fry x1
    18:00-19:30 [workout] Zone-2 cardio 45 min (moderate)
    19:30-20:30 [workout] Push day (hard)
    20:30-21:00 [meal] snack: 1920 Commons: Greek Yogurt Parfait x2
    21:00-08:00 [sleep] Sleep (7.0h min)
  Wednesday
    00:00-08:00 [sleep] Sleep (wake 08:00)
    08:00-08:30 [meal] snack: 1920 Commons: Greek Yogurt Parfait x2
    17:00-17:30 [meal] dinner: 1920 Commons: Chicken & Rice Bowl x2, Hill House: Mediterranean Veggie Bowl x1, KCECH: Salmon with Grains x1, New College House: Tofu Stir-fry x1
    21:00-08:00 [sleep] Sleep (7.0h min)
  Thursday
    00:00-08:00 [sleep] Sleep (wake 08:00)
    07:00-07:30 [meal] breakfast: 1920 Commons: Greek Yogurt Parfait x2
    17:00-17:30 [meal] dinner: 1920 Commons: Chicken & Rice Bowl x2, Hill House: Mediterranean Veggie Bowl x1, KCECH: Salmon with Grains x1, New College House: Tofu Stir-fry x1
    21:00-08:00 [sleep] Sleep (7.0h min)
  Friday
    00:00-08:00 [sleep] Sleep (wake 08:00)
    07:00-07:30 [meal] breakfast: 1920 Commons: Greek Yogurt Parfait x2
    17:00-17:30 [meal] dinner: 1920 Commons: Chicken & Rice Bowl x2, Hill House: Mediterranean Veggie Bowl x1, KCECH: Salmon with Grains x1, New College House: Tofu Stir-fry x1
    21:00-08:00 [sleep] Sleep (7.0h min)
  Saturday
    00:00-08:00 [sleep] Sleep (wake 08:00)
    08:00-08:30 [meal] snack: 1920 Commons: Greek Yogurt Parfait x2
    11:00-11:30 [meal] lunch: Hill House: Mediterranean Veggie Bowl x1, KCECH: Salmon with Grains x1
    17:00-17:30 [meal] dinner: 1920 Commons: Chicken & Rice Bowl x2, New College House: Tofu Stir-fry x1
    21:00-08:00 [sleep] Sleep (7.0h min)
  Sunday
    00:00-08:00 [sleep] Sleep (wake 08:00)
    08:00-08:30 [meal] snack: 1920 Commons: Greek Yogurt Parfait x2
    11:00-11:30 [meal] lunch: Hill House: Mediterranean Veggie Bowl x1, New College House: Tofu Stir-fry x1
    17:00-17:30 [meal] dinner: 1920 Commons: Chicken & Rice Bowl x2, KCECH: Salmon with Grains x1
    21:00-08:00 [sleep] Sleep (7.0h min)

Daily totals:
  Day 0: 3120 kcal, 202g P, 0c
  Day 1: 3120 kcal, 202g P, 0c
  Day 2: 3120 kcal, 202g P, 0c
  Day 3: 3120 kcal, 202g P, 0c
  Day 4: 3120 kcal, 202g P, 0c
  Day 5: 3120 kcal, 202g P, 0c
  Day 6: 3120 kcal, 202g P, 0c
Weekly cost: 0c ($0.00)

Saved schedule figure: /Users/aryanjeena/Downloads/CIS 1921 Project/reports/figures/schedule_lean_bulk_joint_cpsat.png
```

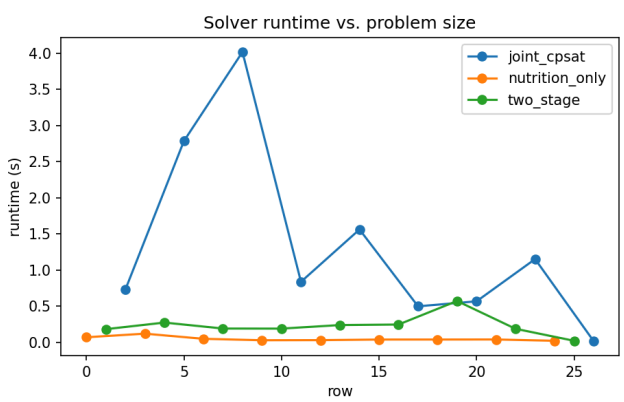


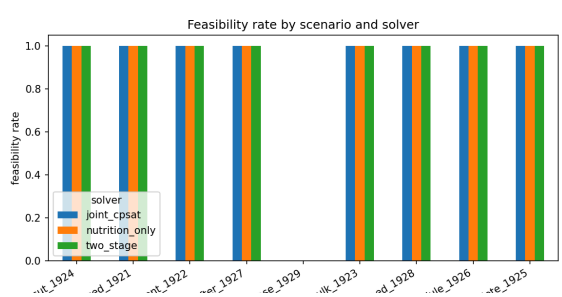
To check that the system fails gracefully, we ran the joint solver on the `infeasible_demo` preset. It returned `INFEASIBLE` in 0.01 seconds with the message "No workout template has a valid start slot inside the user's availability." That is the behavior we expected in the proposal being either a valid plan or a reason why none exists.

```
(base) aryanjeena@lev-15663 CIS 1921 Project %   python -m src.app.cli solve --preset infeasible_demo --solver joint_cpsat --time-limit 10

Solving with joint_cpsat on infeasible_demo (56 foods, 10 workouts)...
Status: INFEASIBLE  runtime=0.01s  objective=None
Infeasibility: No workout template has a valid start slot inside the user's availability (check window lengths vs duration).
```

The full experiment sweep, run with `python -m src.app.cli experiments --time-limit 15 --prefix checkin`, evaluates all nine scenarios against all three solvers for a total of 27 runs. Twenty-four came back `OPTIMAL`, and the other three are all `INFEASIBLE` on `impossible_case` across every solver, which is the expected behavior since that scenario was designed to be unsolvable. The sweep writes two CSVs and five figures to `reports/`. Two observations from the results are worth carrying into the final report. First, joint CP-SAT runs between two and one hundred times slower than nutrition-only but never exceeds five seconds on any feasible instance. Second, `aggressive_cut` produces the largest weekly calorie deviation at 2506, because its calorie band is tight relative to the granularity of the catalog, which makes it a nice example of a constraint that makes instances harder without making them infeasible.

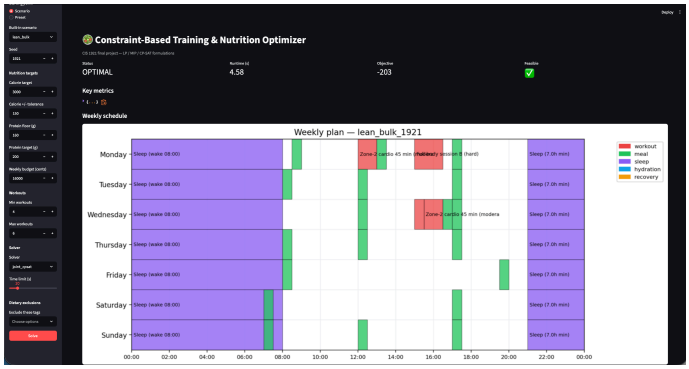
```
(base) aryanjeena@lev-15663 CIS 1921 Project %   python -m src.app.cli experiments --time-limit 15 --prefix checkin

Running 9 instances × 3 solvers (time_limit=15s)...

                instance         solver     status  feasible  runtime_s  objective_value  calorie_deviation_abs  workouts_scheduled
          balanced_1921 nutrition_only    OPTIMAL      True     0.8709          -266.0                    280                   0
          balanced_1921      two_stage    OPTIMAL      True     0.1833             0.0                    280                   3
          balanced_1921    joint_cpsat    OPTIMAL      True     0.7386          -266.0                    280                   3
     budget_student_1922 nutrition_only    OPTIMAL      True     0.1209          -161.0                   1120                   0
     budget_student_1922      two_stage    OPTIMAL      True     0.2743            33.0                   1120                   3
     budget_student_1922    joint_cpsat    OPTIMAL      True     2.7857          -154.0                   1120                   3
         lean_bulk_1923 nutrition_only    OPTIMAL      True     0.8497          -203.0                    840                   0
         lean_bulk_1923      two_stage    OPTIMAL      True     0.1919             0.0                    840                   4
         lean_bulk_1923    joint_cpsat    OPTIMAL      True     4.0172          -203.0                    840                   4
     aggressive_cut_1924 nutrition_only    OPTIMAL      True     0.0298          2506.0                    140                   0
     aggressive_cut_1924      two_stage    OPTIMAL      True     0.1989             0.0                    140                   3
     aggressive_cut_1924    joint_cpsat    OPTIMAL      True     0.8382          2506.0                    140                   3
  vegetarian_athlete_1925 nutrition_only    OPTIMAL      True     0.0313          -182.0                    980                   0
  vegetarian_athlete_1925      two_stage    OPTIMAL      True     0.2399            66.0                    980                   4
  vegetarian_athlete_1925    joint_cpsat    OPTIMAL      True     1.5623          -182.0                    980                   4
tight_class_schedule_1926 nutrition_only    OPTIMAL      True     0.0396          -266.0                    280                   0
tight_class_schedule_1926      two_stage    OPTIMAL      True     0.2474           150.0                    280                   3
tight_class_schedule_1926    joint_cpsat    OPTIMAL      True     0.4991          -266.0                    280                   3
early_morning_lifter_1927 nutrition_only    OPTIMAL      True     0.0398          -266.0                    280                   0
early_morning_lifter_1927      two_stage    OPTIMAL      True     0.5727           200.0                    280                   4
early_morning_lifter_1927    joint_cpsat    OPTIMAL      True     0.5685          -266.0                    280                   4
recovery_constrained_1928 nutrition_only    OPTIMAL      True     0.0407          -266.0                    280                   0
recovery_constrained_1928      two_stage    OPTIMAL      True     0.1808            33.0                    280                   3
recovery_constrained_1928    joint_cpsat    OPTIMAL      True     1.1527          -266.0                    280                   3
     impossible_case_1929 nutrition_only INFEASIBLE     False     0.0228             NaN                      0                   0
     impossible_case_1929      two_stage INFEASIBLE     False     0.0222             NaN                      0                   0
     impossible_case_1929    joint_cpsat INFEASIBLE     False     0.0178             NaN                      0                   0

Figures written to /Users/aryanjeena/Downloads/CIS 1921 Project/reports/figures
Tables written to /Users/aryanjeena/Downloads/CIS 1921 Project/reports/tables
```

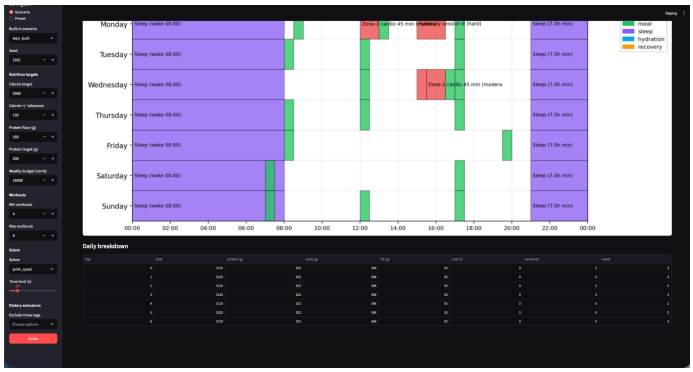




The Streamlit UI launches with `python scripts/launch_ui.py` and is intentionally minimal, since the course document says the frontend can be scrappy and effort should go into the optimization work. The sidebar lets the user pick a scenario or a preset, override every nutrition target, change workout count bounds, toggle dietary exclusions, choose the solver, and set a time limit. After clicking Solve, the main

panel shows status, runtime, objective, and a feasibility badge, followed by a matplotlib weekly-schedule grid and a per-day breakdown table with calories, macros, cost, workout count, and meal count.





Changes to scope:

We have stayed faithful to the proposal's overall goals, but three things changed during implementation and each one reflected a real trade-off rather than a feature being dropped. The first is sleep modeling. The proposal treated sleep as a scheduling variable, but sleep almost always crosses midnight (for example 22:30 to 06:30), and a single-day interval variable on a 48-slot grid cannot cleanly express a wraparound interval. Rather than widen the grid and blow up the variable count, we model sleep as two fixed per-day blocks that are baked into the user's availability mask, so non-overlap with meals and workouts is enforced implicitly and the minimum-hours constraint is still honored. The second change is Penn Dining ingestion. The real pages are JavaScript-rendered, so scraping them properly would need a headless browser, which is a lot of infrastructure for modest gain. We ship a bundled `penn_dining_sample.json` that is semantically identical to what we would scrape, the parser's `fetch()` method is still in place for anyone who extends the pipeline later, and the catalog still includes USDA-style entries and curated sample data, so the real-world-data novelty dimension is still covered. The third change is that hydration reminders are not yet placed as scheduled events. The `HydrationRule` model is in place and used as a soft target count, but placing reminders in the CP-SAT schedule is still to do, and we are tracking it as a pending item on the remaining-tasks list rather than pretending it is done. None of these changes weakens our novelty claim. We still cover real-world data, three comparable solver implementations, a parameterized instance generator, and user-configurable constraints, which is four of the Option 2 bullets instead of the required three.

Remaining tasks:

Six tasks are left. The scaling study comes first as `scripts/run_scaling_study.py` is ready to run, and we will sweep the food catalog over roughly 20, 40, 60, and 80 items and workouts over 5, 10, and 15, then plot runtime against problem size for each solver. This directly answers a proposal question about solver behavior as instances grow, and we expect it to produce some of the best material for the final report. We plan to have this done in the next two days. Next is adding hydration reminders as single-slot events in the joint CP-SAT model with minimum-spacing constraints and a missed-reminder penalty in the objective, which should be on the order of fifty lines of code. After that comes better infeasibility analysis using CP-SAT assumption literals, so that when a model is infeasible we can return a minimal unsatisfiable subset of constraints as a human-readable message instead of a generic status. The largest remaining time cost is the final report itself. A draft outline lives at `reports/draft_outline.md`, and we plan to flesh it into a full write-up covering modeling choices, solver trade-offs, runtime scaling, the feasibility story, and where joint optimization beats two-stage and where they tie. The one-to-two-minute demo video is scheduled for the week of April 21 and will show the CLI demo, a Streamlit session, and the experiment sweep without voiceover. If time allows, we also want to try an LNS warm-start that seeds the joint solver with the two-stage solution as a hint, and add a couple more stress-test presets. We are confident we can finish through the report and video before April 28 without needing any late days.